
PROMPT ENGINEERING REPORT

MediAssist AI

Clinical Decision-Support Prompting with a Hugging Face LLM

Name: Anshul Singh Jadon

Roll No.: 2301920100064

Branch: CSE (4th Year)

Submitted To: Mr. Anant Verma

1. Introduction

MediAssist AI is a prompt-engineering project centered on an open-source, instruction-tuned Hugging Face model. The idea was simple to state but harder to pull off: take a general-purpose 7B language model and, through prompting alone — no fine-tuning — get it to act like a safe, structured, role-aware clinical decision-support assistant. This report walks through how the model was chosen, the prompting techniques used across the seven core tasks and the bonus challenge, the problems that came up along the way, and how the prompts evolved once those problems were addressed.

2. Model Selection

The notebook uses HuggingFaceH4/zephyr-7b-beta as the primary model, loaded via transformers.AutoModelForCausalLM and driven through a single generate_response() wrapper so that every task shares one generation path.

Model	Params	Best Use Case	Advantages	Limitations
HuggingFaceH4/zephyr-7b-beta	7B	Instruction-following chat assistant (used for MediAssist AI)	Ungated/open access; strong instruction-following from DPO fine-tuning; good structured-format compliance	Not medically fine-tuned, can hallucinate; 7B ceiling on complex multi-step reasoning
mistralai/Mistral-7B-Instruct-v0.1	7B	General-purpose instruction tasks; fine-tuning base	Strong base performance; widely benchmarked; efficient (grouped-query attention)	Gated (requires HF access approval); weaker out-of-box instruction adherence than its own DPO-tuned descendant, Zephyr

3. Prompt Engineering Techniques Used

Role / persona prompting — every task anchors the model to a specific professional identity ("clinical decision-support assistant", "board-certified endocrinologist", "triage assistant", "medical data extraction system") rather than asking it as a generic chatbot. This narrows tone, vocabulary, and risk-taking behaviour.

System + user message separation — the chatbot task (Task 1) and the bonus role-aware template both use a dedicated system message via tokenizer.apply_chat_template, keeping the persona/safety rules separate from the user's actual question.

Context grounding — Task 2's diabetes prompt states who the reader is (a newly diagnosed patient with no medical background) before asking for the explanation, and Task 7's optimized prompt supplies the concrete symptom list instead of asking an open-ended question.

Output-format forcing — strict Markdown headings (Task 2), a fixed Symptoms/Severity template (Task 3), an Urgency/Recommendation template (Task 4), and a JSON schema with "no explanation, no code fences" (Task 5) all force the model into a predictable, machine-parseable shape.

Few-shot prompting — Task 3 gives three worked clinical-note-to-summary examples before the real input, teaching the exact output pattern purely by demonstration rather than by instruction.

Chain-of-thought (silent) prompting — Task 4 asks the model to reason step-by-step over the vitals internally but explicitly suppresses that reasoning from the final output, keeping the response clean while still improving the underlying judgment.

Structured data extraction with defensive parsing — Task 5 combines a strict JSON-only instruction with a regex fallback (re.search for the first {...} block) and a try/except around json.loads, so the pipeline degrades gracefully if the model adds stray text.

Prompt optimization / iterative refinement — Task 7 takes a deliberately weak baseline prompt, diagnoses exactly why it fails, and rewrites it with role grounding, symptom grounding, ranked output format, and a fallback instruction for ambiguous cases.

4. Challenges Faced

- **General-purpose base model, not medically fine-tuned:** Zephyr-7B has no clinical training, so every prompt has to compensate with explicit safety rules ("never give a definitive diagnosis", "flag emergencies clearly") rather than relying on the model's own medical judgment.
- **Risk of hallucinated facts and figures:** open-ended medical questions invite invented statistics, so constraints such as "do not invent statistics" and "say so rather than guessing" had to be added directly into the prompt text.
- **Getting reliable structured output from a chat-tuned model:** plain instructions to "return JSON" were not sufficient on their own — the model would sometimes wrap the JSON in prose or code fences, which is why a defensive regex-and-try/except layer was added around the raw generation in Task 5.
- **Balancing brevity with completeness:** without an explicit word cap and heading structure, the diabetes explanation in Task 2 tended to ramble; the fix was a strict word limit plus mandatory Markdown headings.
- **Suppressing visible reasoning without losing its benefit:** Task 4 needed the accuracy benefit of chain-of-thought reasoning but a clean, short final answer for hospital staff, which required an explicit "reason internally but do not show your steps" instruction.
- **Gated vs. ungated model access:** Mistral-7B-Instruct-v0.1 requires Hugging Face access approval, which is a practical barrier for reproducibility, reinforcing the choice of the ungated Zephyr model for the working pipeline.
- **7B-scale reasoning ceiling:** on multi-symptom differential-diagnosis style prompts, a 7B model can still produce shallow or unranked reasoning, which is why Task 7's optimized prompt explicitly forces a ranked list with a one-sentence rationale per item instead of leaving the reasoning implicit.

5. Improvements Made After Prompt Refinement

5.1 Diabetes explanation (Task 2)

Before: "Tell me about diabetes." — no persona, no audience, no structure, no length limit.

After: endocrinologist persona, explicit patient audience, a 300-word cap, plain-language constraint, an instruction to admit uncertainty, and five mandatory Markdown headings (Disease Overview, Symptoms, Risk Factors, Prevention, When to Consult a Doctor).

Result: the response became predictable in length and structure, safe to drop directly into a discharge-summary template, and less prone to invented statistics.

5.2 Differential diagnosis (Task 7)

Before: an ungrounded question produced a poor response — "Maybe malaria. Or maybe dengue. Or maybe COVID. I am not sure." — with no structure, no grounding in real symptoms, and no actionable next step.

After: the rewritten prompt grounds the model in the actual symptom list (fever, joint pain, skin rash, travel to a dengue-endemic region), forbids introducing unrelated conditions, forces a ranked list with a one-sentence rationale per condition, requires a concrete recommended next step (test or referral), and gives the model an explicit fallback instruction for when symptoms are too nonspecific to rank.

Result: the same underlying model moved from an unhelpful, unranked guess list to a structured, evidence-grounded, actionable differential with a clear escalation path.